

Reviewer Pack — Minimal Local Indeterminacy in Noncommutative Geometry and C...

Entry bc0cdba89e7f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 04:16:20 UTC

Minimal Local Indeterminacy in Noncommutative Geometry and Communication Complexity Rank Inequality

Entry ID: bc0cdba89e7f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 04:16:20 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex, d -dimensional noncommutative geometric complex G , the minimal local indeterminacy $\varepsilon(G)$ is linearly correlated with its communication complexity rank $r(G)$, such that $\varepsilon(G) = \Theta(r(G))$. Equivalently: for all instances, if $r(G) \leq k$, then $\varepsilon(G) \leq k^2$.

Rationale (proposer's reasoning):

Noncommutative geometry provides a framework to study the interplay between algebraic structures and geometric properties. By linking this with communication complexity, we may uncover new structures that could potentially explain the difficulty of certain computational problems. The minimal local indeterminacy measures the rigidity of the noncommutative complex, which might be correlated with the complexity of distributing information in a network.

Taxonomy category: Noncommutative Geometry $\times$ Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0244c0be364c0e97

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If for 30 random seeds, the ratio of $\varepsilon(G)$ to $r(G)^2$ exceeds a threshold T , then the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def communication_complexity_rank(G, n):
        # Placeholder function to compute the rank of the communication complexity matrix
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 10)

    def minimal_local_indeterminacy(G, n):
        # Placeholder function to compute the minimal local indeterminacy
        # This is a dummy implementation and should be replaced with actual logic
        return random.random()

    n = random.choice([5, 10, 15, 20, 30, 40])
```

```

G = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
r_G = communication_complexity_rank(G, n)
ε_G = minimal_local_indeterminacy(G, n)

if r_G == 0:
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": 0,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

ratio = ε_G / (r_G ** 2)
return {
    "metric_name": "communication_complexity_rank",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio <= 1.0, # Placeholder threshold
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = len(results) / len(seeds)
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results))
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient=0' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

y_rank', 'metric_value': 0.0020358185279059824, 'instances_tested': 1, 'n_max': 10, 'conjecture_hold
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.025864098919335804, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.008539246253581642, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.006515929276667601, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.018463207014737623, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.0013522347613018348, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.06858598074716979, 'insta
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.0014095810393079077, 'insta

```

TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.013425402709613087, 'inst
 TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.010591473975069779, 'inst
 TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.0008422152570918972, 'ins
 RESULT: SUPPORTED mean=0.06225868624532891 std=0.1752927572332681 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for both ‘communication_complexity_rank’ and ‘minimal_local_indeterminacy’, which are replaced with random values. This does not allow for a meaningful empirical test of the conjecture as it does not compute the actual rank or indeterminacy, making the results irrelevant to the conjecture’s claims.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses placeholder functions for both ‘communication_complexity_rank’ and ‘minimal_local_indeterminacy’, which are replaced with random values. This does not allow for a meaningful empirical test of the conjecture, as it does not compute the actual rank or indeterminacy. The critic’s challenge is valid, and since the pre-registered support condition was not unambiguously met, the verdict is INCONCLUSIVE. | next: Develop a proper test code that computes the actual communication comple

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14417
2	preregistration	ollama_remote	glm4:latest	0	0	14267
3	novelty	ollama_remote	glm4:latest	0	0	20597
4	novelty	ollama_remote	glm4:latest	0	0	19423
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21335
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12019
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13618
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24573
9	critic	ollama_remote	glm4:latest	0	0	12895
10	judge	ollama_remote	glm4:latest	0	0	15766

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 168911 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/bc0cdba89e7f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/bc0cdba89e7f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bc0cdba89e7f.tar.gz` (if generated)